

レポート課題 11

05-191009 奥村真司

問 1

動作例

```
?- ['problem1.pl'].
true.

?- eq(a,c).
ERROR: Out of local stack
```

考察

仮定に以下のように番号をふることにする。

```
1. eq(a,b).
2. eq(c,b).
3. eq(X,Z) :- eq(X,Y),eq(Y,Z).
4. eq(X,Y) :- eq(Y,X).
```

各仮定の論理的な意味は以下ようになる。

- $1. eq(a, b)$
- $2. eq(c, b)$
- $3. \forall X \forall Y \forall Z. eq(X, Y) \wedge eq(Y, Z) \rightarrow eq(X, Z)$
- $4. \forall X \forall Y. eq(Y, X) \rightarrow eq(X, Y)$

仮定 2 より $eq(c, b)$, これと仮定 4 で $X = c, Y = b$ としたものから、 $eq(b, c)$ 。仮定 1 より $eq(a, b)$ であるから $eq(a, b) \wedge eq(b, c)$ 。これと仮定 3 で $X = a, Y = b, Z = c$ としたものより $eq(a, c)$ よって $eq(a, c)$ は true である。

Prolog は番号の若いルールを優先して深さ優先探索によって SLD 導出を探索する。よって探索は以下の画像のように進み、無限ループし、処理系に実際に問い合わせるとスタックオーバーフローした。論理的解釈と一致させるには、処理系の探索方法を幅優先探索にすればよい。実際発展 2 の処理系では true. が表示される。

問 2

```
?- ['problem2.pl'].
true.
```

```
?- test.  
true.  
  
?- q(X,X).  
X = f(X).
```

考察

論理的解釈では、 $q(X,X)$ と $q(X,f(X))$ は単一化できないので、false となると考えられる。しかし動作例のように `test.` を Prolog 処理系に問い合わせると true となる。これは、Prolog 処理系が単一化の際に出現検査をしていないからであり、 $q(X,X)$ を問い合わせると動作例のように $X=f(X)$ と出力される。この出現検査の有無が論理的解釈との差異を生んでいる。

問 3

```
?- ['problem3.pl'].  
true.  
  
?- \+p(X),q(X).  
false.  
  
?- q(X),\+p(X).  
X = b.
```

考察

閉世界仮説より、 $\forall X.(p(X) \leftrightarrow X = a), \forall X.(q(X) \leftrightarrow X = b)$ であるから、 $\neg p(b)$ は true で $q(b)$ も true, すなわち $\neg p(b) \wedge q(b)$ が true なので $\exists X.(\neg p(X) \wedge q(X))$ は論理的解釈では true と考えられる。

Prolog 処理系で実際に問い合わせると、動作例のように false が帰ってくる。Prolog 処理系での否定は導出の失敗による否定で、変数が含まれる場合にもこれが適用される。処理系に問い合わせると、 $p(X)$ が導出できるかどうかを探索するが、 $p(a)$ が true で導出できるので $\forall +p(X)$ は false となる。よって $\forall +p(X), q(X)$ も false となる。このように、Prolog 処理系は内部に変数を含む項にたいしても失敗による否定を適用することが論理的解釈との差異を生んでいる。

論理的解釈と結果を一致させるためには Prolog がゴールを解決しようとするときにそれに含まれる変数がすべて変数を含まないものに置き換わっているようにすればよいので、問い合わせを $q(X), \forall +p(X)$ に書き換えると、動作例のように true. となる。この場合は探索中に X は b に置き換わり、ゴールの導出に成功する。

発展 1

動作例

```
?- ['advanced1.pl'].
true.

?- r(a).
ERROR: Out of global stack
```

考察

論理的解釈では、 $p(a)$ は true もしくは false のどちらかである。 $p(a)$ が true の場合は仮定 1 より、false の場合は仮定 2 より、いずれの場合も $r(a)$ は true となるので、 $r(a)$ が true は論理的帰結である。

$r(a)$ を問い合わせると、処理系はルール 1 より $p(a)$ の導出を試みる。しかしゴールは $p(a) \rightarrow p(f(a)) \rightarrow p(f(f(a))) \rightarrow p(f(f(f(a)))) \rightarrow \dots$ となり無限ループする。Prolog の否定は失敗による否定で、 $p(a)$ の導出に成功すれば $\neg p(a)$ の導出は失敗し、 $p(a)$ の導出に失敗すれば $\neg p(a)$ の導出は成功する。しかし $p(a)$ の導出が成功も失敗もせず無限ループした場合、 $\neg p(a)$ の導出は成功も失敗もしない。

発展 2

動作例

```
?- rule eq(a,b).
?- rule eq(c,b).
?- rule eq(X,Z) :- eq(X,Y),eq(Y,Z).
?- rule eq(X,Y) :- eq(Y,X).
?- query eq(a,c).
true.
-----
?- rule test :- q(X,X).
?- rule q(X,f(X)).
?- query test.
false.
```

考察

基本的には前回の課題の発展 1 のインタプリタである。否定は実装していない。(採点の担当の方が先週と違う場合を考えてこのレポートの最後の部分に前回の発展 1 のレポートを引用しております。適宜参照していただけるとありがたいです。) 以下で論理的に健全で完全であることを説明する。Prolog 処理系とことなり深さ優先探索ではなく幅優先探索をしており、証明 (木) は有限の深さであるから証明が存在するならば、必ず有限時間でそれを発見し報告できる。幅優先探索にすることで自然にルールの優先順位が取り払われている。ここでここまでの方法と SLD 導出の定義のスライドと見比べたときに異なる点は”ゴールリストのサブゴールを 1 つ選ぶ”という点で、先週の処理系ではかならずゴールリストの先頭を選ぶようになっていたのでここが任意の一つ選ばれるように改良した。今回の問 2 で見たように、単一化の際の出現検査をしないと論理的解釈と差異が生じるので、単一化の出現検査をちゃんとやるようにしている。よって SLD 導出の健全性、完全

性より今回実装した Prolog ライクな論理型言語は完全かつ健全ある。

動作例では, 問 1, 問 2 で Prolog では論理的解釈と差異が生じていたものが、今回実装したものでは差異を生じないことが確認できる。

前回の発展 1(前回分レポートより引用)

動作例

```
?- rule nat(z).
?- rule nat(s(X)) :- nat(X).
?- query nat(X).
X = z
;
X = s(z)
;
X = s(s(z))
;
X = s(s(s(z)))
;
X = s(s(s(s(z))))
;
X = s(s(s(s(s(z)))))
;
X = s(s(s(s(s(s(z))))))
;
X = s(s(s(s(s(s(s(z)))))))
.
?- rule male(koji).
?- rule parent(kobo,koji).
?- rule father(X,Y) :- parent(X,Y),male(Y).
?- query father(kobo,Z).
Z = koji
;
false.
?- rule add(z,Y,Y).
?- rule add(s(X),Y,s(Z)) :- add(X,Y,Z).
?- query add(s(z),X,X).
false.
?- rule append([],Y,Y).
?- rule append([A|X],Y,[A|Z]) :- append(X,Y,Z).
?- rule concat([X|[]],X).
?- rule concat([X|Y],Z) :- concat(Y,W),append(X,W,Z).
?- query append([1,2,3],[4,5],Z).
Z = [1, 2, 3, 4, 5]
;
false.
?- query append(X,Y,[1,2,3,4,5]).
Y = [1, 2, 3, 4, 5]
X = []
;
Y = [2, 3, 4, 5]
X = [1]
;
Y = [3, 4, 5]
X = [1, 2]
;
Y = [4, 5]
X = [1, 2, 3]
;
```

```

Y = [5]
X = [1, 2, 3, 4]
;
Y = []
X = [1, 2, 3, 4, 5]
;
false.
?- query concat([[1],[2,3,4]] ,X).
X = [1, 2, 3, 4]
;
false.
?- rule in(X,[X|Y]).
?- rule in(X,[Y|Z]) :- in(X,Z).
?- rule choose([X|Y],Y,X).
?- rule choose([A|X],[A|Y],Z) :- choose(X,Y,Z).
?- rule hamilton(V,E) :- choose(V,Rem,Start),hamiltonsub(Rem,E,Start).
?- rule hamiltonsub([],X,Y).
?- rule hamiltonsub(V,E,Now) :- choose(V,Rem,Next),in([Now|[Next|[]]],E),
hamiltonsub(Rem,E,Next).
?- query hamilton([1,2,3,4],[[1,2],[2,3],[3,4],[4,1]]).
true.
;
true.
;
true.
;
true.
;
false.
?- query abcd.
Undefined procedure.
?- query a([,
Parse Error.
?- $
Failure Unknown Token: $

```

考察

Prolog ライクな論理型言語のインタプリタをレクサー、パーサーも含め実装した。動作例のように、前回の授業で扱ったものや課題の解答で自分で作成したものについて、期待通りに動作することがわかった。add の例では出現検査をちゃんと行えていることが確認できた。この動作例における入力データは verify.pl 内にある。

定数、項、述語などの扱い

実装を始めるにあたってまず定数、項、述語などのデータをどう扱うかを考えた。これらの文法は Prolog と同じようにした。syntax.ml 内でこれらの型は下のように定義されている。

```

type name = string
type var = int

type term =
  | EConst of name
  | EFnSymb of name * (term list)
  | EVar of name

```

```

| ECons of term * term
| ENil

type fact = EPdSymb of name * (term list)

type rule = fact * (fact list)

type command =
| CRule      of rule
| CQuery     of fact list
| CCont
| CExit

type subst = (name * term) list

type constraints = (term * term) list

```

term が授業スライドでの項にあたるもので、定数が EConst, ファンクタが EFnSymb(Function Symbol),... というようになっている。述語 (項, 項,...) を fact という名前にした (この形式に fact という名前をつけただけでこれが意味的に fact とは限らない。例えば問い合わせでのゴールのリストは fact list である。)。述語 (項, 項,...) :- 述語 (項, 項,...), 述語 (項, 項,...),... が rule である。

ルールの読み込みと問い合わせの区別は入力先頭に rule か query とつけることを要求することで実現した。query ではそこまでに入力された rule 全てを用いて SLD 導出を探索する。

探索

次に探索について説明する。授業スライドにもあるとおり SLD 導出を探索しているが、swi-prolog と異なり幅優先探索 (BFS) で探索し、単一化時に出現検査を行うようにした。

ゴールのリストの先頭とルールを単一化する際に、入力された通りの変数名を用いると変数名がかぶっているときに正しく単一化できないので、ルールを用いるたびに内部の変数名をすべて unique なものに置き換えている。が syntax.ml 内の convert_*** という関数群はこの置き換えを行うための補助関数である。例えば `father(X,Y) :- parent(X,Y),male(Y)` という rule なら、`father(_1,_2) :- parent(_1,_2),male(_2)` のように置き換えられる。同じルールでも置き換えを行うたびに別のものになる。

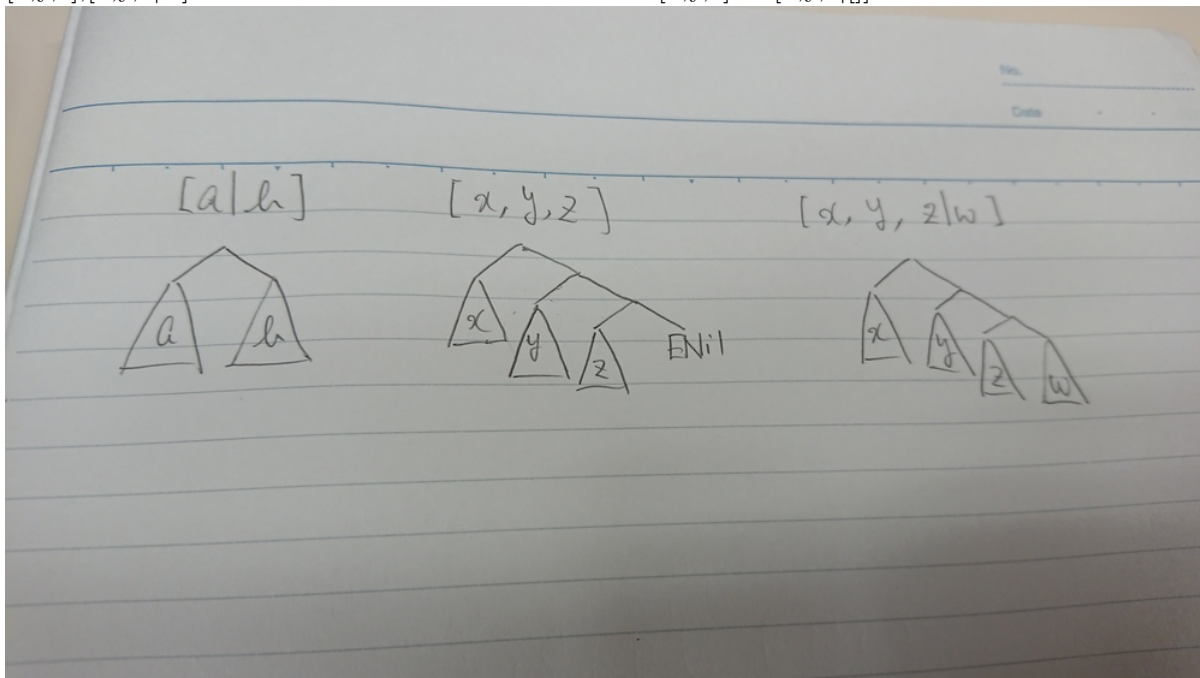
探索の各状態は fact list 型のゴールのリストと subst 型の置き換えの組になっている。BFS の実現方法について説明する。BFS は Queue を用いて行った。計算量的には好ましくないが、list を queue とみなし @ で後ろにつなげることで push を、先頭の取り出しはパターンマッチで行っている。eval 内の next という関数は探索状態と仕様するルールを受け取って、ゴールリストの先頭とルールが単一化可能なら次の状態の singleton リストを、不可能なら空リストを返す関数であり、`List.concat (List.map (fun r -> next q r) rules)` とすることで現在の状態 q からルール rules を用いて遷移可能な状態のリストが得られる。これを queue の後ろに @ でつないでいる。これを @ で前につなぐように変更するだけで簡単に DFS にすることもできる。解が見つかったらその時点での queue の状態も一緒に返してやることで、続きからの探索も可能になっている。

その他こだわりポイント

以下は課題の出題意図とは直接的には関係ないと思われるがこだわった点について説明する。

- リストの $[x,y,z]$ の入出力形式のサポート

はじめはコンスの形のみサポートしていたが非常に入力しづらいので $[x,y,z]$ の形式をサポートした。これが意外と思考を要したのでそれについて説明する。 $[a|b]$ を以下の画像のように捉えることで、term の構造は ECons による完全二分木と捉えることができる。 $[x,y,z],[x,y,z|w]$ は下のような木構造を表すことから、 $[x,y,z]$ は $[x,y,z|[]]$ の省略形と思える。



よって、最後の要素で場合分けして考えることで Parser は以下のように書けた。

```
atomic:
  | SYMBID { EConst($1) }
  | NUMBER { EConst($1) }
  | VARID  { EVar($1) }
  | list   { $1 }
;

atomics:
  | list_last { $1 }
  | atomic COMMA atomics { ECons($1,$3) }
;

list_last:
  | atomic { ECons($1,ENil) }
  | atomic BAR atomic { ECons($1,$3) }
;

list:
  | NIL { ENil }
  | LBRACKET atomics RBRACKET { $2 }
;
```

出力も、カンマを出力すべきかどうかをパターンマッチで場合わけして以下のように書くことができた。

```
let rec print_term t =
  match t with
  | EFnSymb (id,t1) -> print_name id;
```

```

                                if tl=[] then ()
                                else (print_string "(";
                                        print_term_list tl;
                                        print_string ")")

| EVar id -> print_name id
| ECons (x,y) -> print_list t
| ENil -> print_string "[]"
| EConst x -> print_name x
and print_term_list tl =
  match tl with
  | [] -> ()
  | t::rest -> print_term t;
               (if (rest <> []) then (print_string ",")
               else ());
               print_term_list rest
and print_list t =
  print_string "[";
  print_list_inner t;
  print_string "]"
and print_list_inner t =
  match t with
  | ECons (x,(ECons(y,z))) -> print_term x;
                              print_string ", ";
                              print_list_inner (ECons (y,z))
  | ECons (x,ENil) -> print_term x
  | ECons (x,y) -> print_term x;
                  print_string " | ";
                  print_term y
  | _ -> () (* Error *)

```

- エラー

動作例の最後から3つめのように、全く定義されていない述語についての問い合わせが来た場合に、閉世界仮説的には false. でいいのではないかと思ったが、swi-prolog では Undefined procedure というエラーが出るようになっていたのでそれを採用し、チェックするようにした。動作例の最後のあたりのように、レクサーやパーサーのエラーも含めエラーが発生しても処理系はエラーメッセージを出力して次の入力を待ち、終了しないようにした。なおエラー出力および false. は bash で赤文字で出力されるようにした。